ros2_tracing: Multipurpose Low-Overhead Framework for Real-Time Tracing of ROS 2

Christophe Bédard, Ingo Lütkebohle, and Michel Dagenais, *Senior Member, IEEE*

***Abstract*—Testing and debugging have become major obstacles for robot software development, because of high system complexity and dynamic environments. Standard, middleware-based data recording does not provide sufficient information on internal computation and performance bottlenecks. Other existing methods also target very specific problems and thus cannot be used for multipurpose analysis. Moreover, they are not suitable for real-time applications. In this letter, we present `ros2_tracing`, a collection of flexible tracing tools and multipurpose instrumentation for ROS 2. It allows collecting runtime execution information on real-time distributed systems, using the low-overhead LTTng tracer. Tools also integrate tracing into the invaluable ROS 2 orchestration system and other usability tools. A message latency experiment shows that the end-to-end message latency overhead, when enabling all ROS 2 instrumentation, is on average 0.0033 ms, which we believe is suitable for production real-time systems. ROS 2 execution information obtained using `ros2_tracing` can be combined with trace data from the operating system, enabling a wider range of precise analyses, that help understand an application execution, to find the cause of performance bottlenecks and other issues.**

***Index Terms*—Software tools for robot programming, distributed robot systems, Robot Operating System (ROS), performance analysis, tracing.**

I. Introduction

As MODERN robots have become more versatile, e.g., in tackling unstructured environments or collaborative work, their software has become correspondingly more complex. Distributed, asynchronous compute graphs based on frameworks like ROS [1], [2] are now the dominant approach for integrated systems, even for space exploration [3]. Correspondingly, testing and debugging is now typically conducted by collecting data from a running system for later analysis, through tools like rosbag or textual logging [4], [5].

Manuscript received January 3, 2022; accepted April 21, 2022. Date of publication May 11, 2022; date of current version May 23, 2022. This letter was recommended for publication by Associate Editor T. Horii and Editor T. Ogata upon evaluation of the reviewers' comments. This work was supported in part by the European Union's Horizon 2020 research and innovation programme under Grant 780785. The work of Christophe Bédard and Michel Dagenais was supported in part by Ericsson, in part by NSERC, and in part by Prompt. *(Corresponding author: Christophe Bédard.)*

Christophe Bédard and Michel Dagenais are with the Department of Computer Engineering and Software Engineering, Polytechnique Montréal, Montreal, QC H3T 1J4, Canada (e-mail: christophe.bedard@polymtl.ca, bedard.christophe@gmail.com; michel.dagenais@polymtl.ca).

Ingo Lütkebohle is with Corporate Research at Robert Bosch GmbH, 71272 Renningen, Germany (e-mail: ingo.luetkebohle@de.bosch.com).

The source code is available at: https://gitlab.com/ros-tracing/ros2_tracing.

Digital Object Identifier 10.1109/LRA.2022.3174346

However, there are well-known drawbacks: rosbag and similar middleware-based tools can only record data that is available as messages. Aside from the effort involved, there is also a significant resource cost in both CPU and memory usage. It is also well known that perturbing a system through extensive monitoring is to be avoided [6]. Therefore, messages simply cannot practically deliver the stacktrace-level of detail and the detailed execution context information that we have come to expect from classical debuggers. Logging also cannot fill this gap, because of its unstructured output and lack of support for binary data. As a result, the debugging experience in robotics is greatly impoverished.

In contrast, tracing has been developed to provide structured, flexible, on-demand data capture across multiple applications and the kernel, to enable detailed analysis when needed. Conceptually, it can be considered an evolution of logging with support for binary data and well-defined data structures. Common frameworks provide support for easy and low-overhead capture of contextual data, such as process information or accurate, in-process timestamps, as well as aggregation of data across hosts and tooling for analysis [7]–[10].

However, two challenges need to be solved to truly improve the testing and debugging situation. First, tracing has so far been a tool for performance experts, used in very specific analysis use-cases – such as scheduling optimization [11] or message latency analysis [12] – that were difficult to extend. To improve the debugging experience in general, a more versatile approach is required. Second, we need to ensure that the performance requirements of robotics are met. Standard ROS 2 targets *soft* real-time systems, i.e., systems where reaction time should have an upper bound, and should be achieved almost always, even though exceeding the bound is not catastrophic. To maintain these characteristics, a tracing integration must have comparatively low overhead with very few outliers.

Contributions: In this letter, we present `ros2_tracing`, a framework for tracing ROS 2 [2] with a collection of multipurpose low-overhead instrumentation and flexible tracing tools. This new tool enables a wider range of precise analyses that help understand an application execution. The source code is available at: https://gitlab.com/ros-tracing/ros2_tracing. `ros2_tracing` brings the following contributions:

- It offers extensible tracing instrumentation capable of providing execution information on multiple facets of ROS 2.
- With a strategic two-phase instrumentation design and using a low-overhead tracer, it has a lower runtime overhead

than current solutions, making it suitable for the real-time applications targeted by ROS 2.
- It enables more precise analyses using combined ROS 2 userspace and kernel space data as a whole.

Furthermore, notable `ros2_tracing` features include a close integration into the expansive suite of ROS 2 orchestration & general usability tools, and an easily swappable tracer backend to support different operating systems or to switch to a tracer with other desired features.

This letter is structured as follows: We survey related work in Section II and summarize relevant background information in Section III. We then present our solution in Section IV and discuss its analysis potential in Section V. Thereafter, Section VI presents an evaluation of the runtime overhead of our solution. Future work is outlined in Section VII. Finally, we conclude in Section VIII.

II. Related Work

Tracing is an established approach for performance analysis, popular for operating system-level performance analysis and for distributed systems. Its popularity is both due to the low overhead when not in use, which is often zero, and due to the extensive tool support. An excellent overview, covering both cloud and operating system use-cases, is [6], [13]. However, while powerful, these tools arguably operate at an abstraction level that is too low to be practical for the average roboticist.

In the context of robotics and ROS 1, the earliest reported use of tracing was motivated by non-deterministic behavior of obstacle avoidance in a mobile robot, by one of the present authors [14]. Using a model of the ROS 1 navigation stack, and tracing based on LTTng [8], a lack of synchronization between the sensory data processing and motion control pathways could be identified. This is a primary example of how non-deterministic effects can be hard to diagnose otherwise, since the magnitude of the effect was dependent on how the OS scheduled the threads involved, which also varied over the run-time of the system. There were some attempts at generalizing this kind of tracing tooling in [15], and deriving a message flow analysis [16]. However, they were discontinued due to the emergence of ROS 2 and its potential for real-time applications [17], [18].

Previous work has identified various open problems in ROS 2. Kronauer *et al.* [19] investigated the end-to-end latency of communications and found that its overhead is up to 50% compared to directly using DDS, the underlying middleware. Similarly, Jiang *et al.* [20] found that the message conversion cost is highly dependent on the complexity of the message structure. Casini *et al.* [21] proposed a scheduling model that aims to bound the end-to-end latency of processing chains. Furthermore, several previous contributions propose tools that use tracing to measure and/or improve message transmission latency, due to its importance for realizing low-latency distributed systems. This includes the RAPLET tool by Nishimura *et al.* [12] for ROS 1, ROS-FM by Rivera *et al.* [22], and ROS-Llama by Blass *et al.* [11], both targeting ROS 2. The last two are monitoring tools, meaning that they use instrumentation to extract execution information, process it, and provide the results to users or act on them during runtime. However, none of these tools support any use-cases beyond latency, and they all show significant overheads (cf. Table I), which is due for some of them to the use of logs or custom unoptimized tracers to extract execution information. An interesting custom proposal to detect latency deadline violations with a low overhead of only 86 μs per event, including online detection, is proposed by Peeck *et al.* [23], but again, without attempt at generality.

In conclusion, the present work is – to the best of our knowledge – the first and only tool that provides a generic, tracing-based approach for ROS 2 performance analysis.

III. Background

In this section, we summarize relevant background information needed to support subsequent sections. Note that we use "ROS 1" to refer to the first version of ROS and use "ROS" to refer to ROS 1 and ROS 2 in general, since many concepts apply to both.

A. ROS 2 Architecture

The ROS 2 architecture has multiple abstraction layers; from top to bottom, or user code to operating system: `rclcpp`/`rclpy`, `rcl`, and `rmw`. The user code is above and the middleware implementation is below. `rclcpp` and `rclpy` are the C++ and Python client libraries, respectively. They are supported by `rcl`, a common client library API written in C which provides basic functionality. The client libraries then implement the remaining features needed to provide the application-level API. This includes implementing executors, which are high-level schedulers used to manage the invocation of callbacks (e.g., timer, subscription, and service) using one or more threads. `rmw` is the middleware abstraction interface. Each middleware that is used with ROS 2 has an implementation of this interface. Multiple middleware implementations can be installed on a system, and the desired implementation can be selected at runtime through an environment variable, otherwise the default implementation is used. As of ROS 2 Galactic Geochelone, the default middleware is Eclipse Cyclone DDS [24].

B. ROS Nodes and Packages

ROS is based on the publish-subscribe paradigm and also supports the RPC pattern under the "service" name. ROS nodes may both publish typed messages on topics and subscribe to topics, and they can use and provide services. While the granularity and semantics of nodes in a system is a design choice, the resulting node and topics structure is analogous to a computation graph.

There are also specialized nodes called "lifecycle nodes" [25]. They are stateful managed nodes based on a standard state machine. This makes their life cycle easier to control, which can be beneficial for safety-critical applications [18]. Node life cycles can also be split into initialization phases and runtime phases, with dynamic memory allocations and other non-deterministic actions being constrained to the initialization phases for real-time applications.

TABLE I
SUMMARY OF EXISTING MONITORING AND INSTRUMENTATION METHODS AND COMPARISON WITH PROPOSED METHOD

Method	Type[a]	ROS 1 / 2	Instrumentation					Extensible	Launch tools	Overhead[b]	Source avail.
			messages	callbacks	services	executor	lifecycle				
ROS-FM [22]	M	1 / 2	✓	×	✓	- / ×	- / ×	×	×	15-515% C	×
tracetools [15], [16]	I	1	✓	✓	×	-	-	×	×	?	✓
RAPLET [12]	I	1	✓	✓	×	-	-	×	×	2-20% L	✓
ROS-Llama [11]	M	2	✓	✓	×	✓	×	×	×	30-40% C	×
`ros2_tracing`	I	2	✓	✓	✓	✓	✓	✓	✓	1-15% L	✓

[a] M and I for monitoring and instrumentation-only types, respectively.
[b] C and L for CPU and latency overhead, respectively.

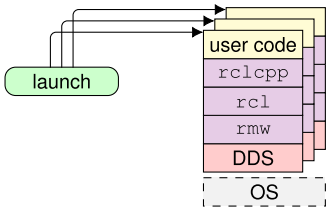

Fig. 1. Overall ROS 2 architecture and tooling interaction.

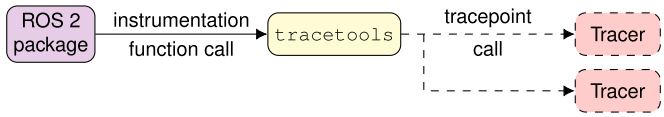

Fig. 2. Instrumentation and tracepoint calls.

As for the code, in ROS, it is generally split into multiple packages, which directly and indirectly depend on other packages. Each package has a specific purpose and may provide multiple libraries and executables, with each executable containing any number of nodes. The ROS ecosystem is federated: packages are spread across multiple code repositories, on various hosts, and are authored and maintained by different people. The ROS 2 source code itself is made up of multiple packages that approximately match its architecture.

C. Usability and Orchestration Tools

Much like ROS 1, ROS 2 has many tools for introspection, orchestration, and general usability. There are various `ros2` commands, including `ros2 run` to run an executable and `ros2 topic` to manually publish messages and introspect published messages. Packages can also provide extensions that add other custom commands. The `ros2 launch` command is the main entry point for the ROS 2 orchestration system and allows launching multiple nodes at once. This is configured through Python, XML, or YAML launch files which describe the system to be launched using nodes from any package or even other launch files. Since ROS systems can be quite complex and contain multiple nodes, an orchestration system is indispensable. Launch files can also be used to orchestrate test systems and verify certain behaviors or results.

D. Generalizability

Fig. 1 shows a summary of the ROS 2 architecture and the main tooling interaction. The architecture and launch system can be generalized down to an orchestration tool managing an application layer on top of a middleware. Therefore, the tool presented in this letter could be applied to other similar robotic systems.

IV. `ros2_tracing`

In this section, we present the design and content of `ros2_tracing`. It contains multiple ROS packages to support three different but complementary functionalities: instrumentation, usability tools, and test utilities. Table I shows a comparison between our proposed method and the existing methods mentioned in Section II.

A. Instrumentation

As shown in Fig. 2, core ROS 2 packages are instrumented with function calls to the `tracetools` package. This package provides tracepoints for all of ROS 2 and is the one that triggers them. Tracepoints usually act as instrumentation points and could be directly added to the instrumented code. This creates an indirection, which we introduced for two complementary reasons. First, it allows for abstracting away the tracer backend and allows to easily switch the tracer. Indeed, by replacing the compiled `tracetools` library, the tracer backend can be replaced without affecting the instrumented packages (i.e., the core ROS 2 code). This could be done to support tracing on a different platform or to use a tracer that has other desired functionalities. Second, this keeps instrumented packages free of boilerplate code (e.g., tracepoints definitions and other required preprocessor macros). However, the main advantage of this design choice also has a slight downside, since adding new tracepoints requires modifying both the instrumented package and the `tracetools` package.

As shown in Table I, in addition to being extensible, our method provides instrumentation for multiple aspects of ROS 2, including messages, callbacks, services, executor states, and lifecycle states. Table II presents a full list of the instrumentation points provided by `tracetools`. We split the instrumentation points into two types: initialization events and runtime events. The former collect one-time information about the state of objects, e.g., creation of publishers, subscriptions, and services. The latter collect information about events throughout the runtime, e.g., message publication and callback execution. The former are therefore predominantly triggered during the system

TABLE II
INSTRUMENTATION POINTS LIST WITH TYPES

ROS 2 Layer	Instrumentation Point Name	Type[a]	Note[b]
rclcpp	rclcpp_subscription_init	I	
	rclcpp_subscription_callback_added	I	
	rclcpp_publish	R	P
	rclcpp_take	R	S
	rclcpp_service_callback_added	I	
	rclcpp_timer_callback_added	I	
	rclcpp_timer_link_node	I	
	rclcpp_callback_register	I	
	callback_start	R	S
	callback_end	R	
	rclcpp_executor_get_next_ready	R	S
	rclcpp_executor_wait_for_work	R	S
	rclcpp_executor_execute	R	S
rcl	rcl_init	I	
	rcl_node_init	I	
	rcl_publisher_init	I	
	rcl_subscription_init	I	
	rcl_publish	R	P
	rcl_take	R	S
	rcl_client_init	I	
	rcl_service_init	I	
	rcl_timer_init	I	
	rcl_lifecycle_state_machine_init	I	
	rcl_lifecycle_transition	R	
rmw	rmw_publisher_init	I	
	rmw_subscription_init	I	
	rmw_publish	R	P
	rmw_take	R	S

[a]I and R for initialization and runtime types, respectively.
[b]P and S for publishing and message reception hot paths, respectively.

initialization phase and are used to minimize the payload size of the latter. This strategic instrumentation design is key to minimizing the overhead in the runtime phase. For example, all publisher-related tracepoints in the runtime phase include a unique identifier for the publisher, which is then matched with the data collected by the publisher-related tracepoints in the initialization phase (e.g., topic name, corresponding node name, etc.), thus minimizing the payload size of runtime tracepoints. The information that is collected to form the trace data can then be used to build a model of the execution. Due to the very abstractional nature of the ROS 2 architecture, multiple instrumentation points are sometimes needed to gather the necessary information. For example, to build a model of a subscription, we collect information about its callback function from `rclcpp` and its topic name from `rcl`. This is because callbacks are managed by the client library. Furthermore, both the instrumentation point name and the payload are meaningful: some instrumentation points only differ by their names and are used to indicate the originating layer. Since the instrumentation points cover multiple analysis use-cases, if a portion of the information is not needed for a given analysis, some of the instrumentation points can be disabled and therefore have virtually no impact on execution.

We did not instrument the Python client library, since it is not used for the kind of real-time applications that we are considering with `ros2_tracing`. However, we instrumented `rcl` directly whenever possible. By putting the instrumentation as low as possible in the ROS 2 architecture, it can be leveraged to more easily support other client libraries in the future.

The *Linux Trace Toolkit: next generation* (LTTng) tracer was chosen as the default tracing backend, for its low overhead and real-time compatibility as well as its ability to trace both the kernel and userspace [8], [26]. The runtime cost per LTTng userspace tracepoint on a vanilla Linux kernel using an Intel i7-3770 CPU (3.40 GHz) with 16 GB of RAM is approximately 158 ns [7]. Since it is a Linux-only tracer, all instrumentation calls to the `tracetools` package are preprocessed out on other platforms. This can also be achieved on Linux through a build option.

B. Usability Tools

In line with the ROS 2 usability and orchestration tools, our proposed solution includes two different interfaces to control tracing: a `ros2 trace` command and a `Trace` action for launch files. The `ros2 trace` command is a simple command that allows configuring the tracer to start tracing. The system or executable to be traced must then be run or launched in a separate terminal. When the application is done running, tracing must be stopped in the original `ros2 trace` terminal. On the other hand, the `Trace` action can be used in XML, YAML, and Python launch files. It then allows configuring the tracer and launching the system at the same time. Tracing is stopped automatically after the launched system has shut down, either on its own or after being manually terminated. Listing 1 shows an example with an XML file that launches two nodes. While ROS 2 does not currently natively support it, this would be useful for remote or multi-host orchestration to trace all hosts at once and aggregate the resulting traces.

Listing 1
`Trace` ACTION IN XML LAUNCH FILE

```
<launch>
  <trace
    session-name="ros2" events-ust="ros2:*"/>
  <node pkg="package_a" exec="executable_x"/>
  <node pkg="package_b" exec="executable_y"/>
</launch>
```

Furthermore, these tools can be used to leverage existing LTTng instrumentation (e.g., kernel and other userspace instrumentation) and to enable any custom application-level tracepoints to record other relevant data. For example, LTTng provides shared libraries that can be preloaded with `LD_PRELOAD` to intercept calls to `libc`, `pthread`, the dynamic linker, and function entry & exit instrumentation (added with `-finstrument-functions`) and trigger tracepoints before calling the real functions. If those tracepoints are enabled through launch files, the corresponding shared libraries will be located and preloaded automatically for all executables, which greatly simplifies the launch configuration. The tools also do not prevent users from directly configuring the tracer for advanced options. They are only a thin flexible compatibility and usability layer for ROS 2 and use the LTTng Python bindings for tracer control.

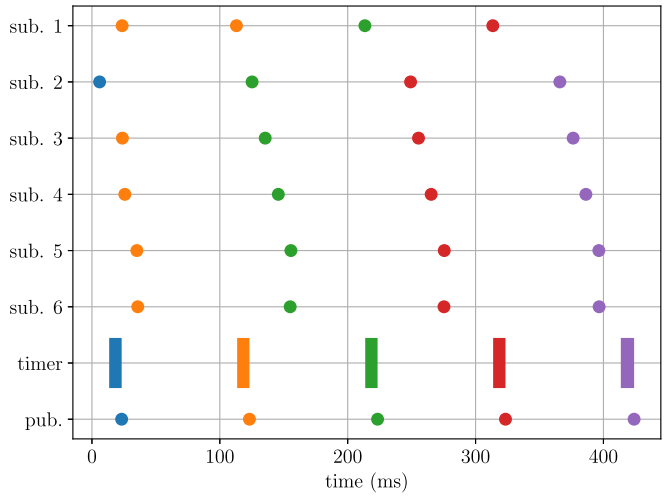

Fig. 3. Example time chart of subscription message reception (sub.), timer callback execution (timer), and message publication (pub.). Message reception and publication instances are displayed as single timestamps, while timer callback executions are displayed as ranges, with a start and an end. The periodic timer callback uses the last received message from each subscription to compute a result and publish it; this inputs-outputs link is illustrated using colors, highlighting an inadequate synchronization.

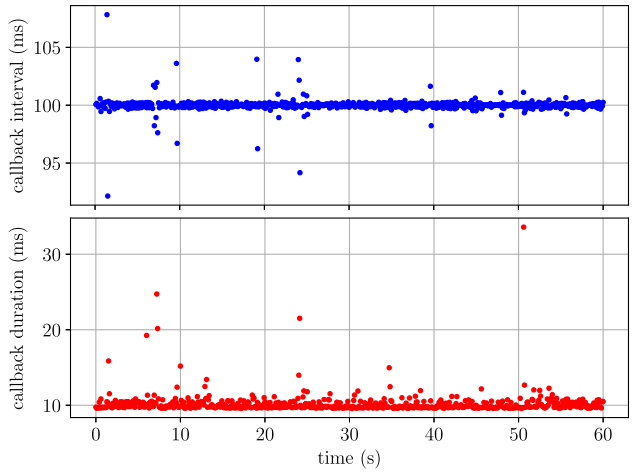

Fig. 4. Example timer callback execution interval (top) and duration (bottom) over time. The callback period is set to 100 ms, while the callback duration depends on the work done. Both contain outliers.

C. Test Utilities

The `tracetools_test` package provides a test utility that allows running nodes and tracing them. The resulting trace can then be read in the test using the `tracetools_read` package to assert results or behaviors in a lower-level, less invasive way.

V. Analysis

The instrumentation and tracing tools provided by `ros2_tracing` allow collecting execution information at the ROS 2 level. This information can then be processed using existing tools to compute metrics or to provide models of the execution. For example, we traced a ROS 2 system that simulates a real-world autonomous driving scenario [27]. In this example, a node receives data from 6 different topics using subscriptions. When the subscriptions receive a new message, they cache it so that the node only keeps the latest message for each topic. The periodically-triggered callback uses the latest message from each topic to compute a result and publish it. To analyze the trace data, we wrote a simple script using `tracetools_analysis` [28], a Python library to read and analyze trace data. As shown in Fig. 3, we can display message reception and publication instance timestamps as well as timer callback execution intervals over time. There is a visible gradual time drift between the input and output messages, which could impact the validity of the published result, similar to the issue described by [14]. This could warrant further analysis and tuning, depending on the system requirements. We can also compute and display the timer callback execution interval and duration, as shown in Fig. 4. The timer callback period is set to 100 ms and the duration is approximately 10 ms, but there are outliers. This jitter could negatively affect the system; these anomalies could warrant further analysis as well.

To dig deeper, this information can be paired with data from the operating system: OS trace data can help find the cause of performance bottlenecks and other issues [30]. Since ROS 2 does higher-level scheduling, this is critical for understanding the actual execution at the OS level. Using LTTng, the Linux kernel can be traced alongside the application. The ROS 2 trace data that was obtained using the `ros2_tracing` instrumentation can be analyzed together with the OS trace data using Eclipse Trace Compass [29], which is an open-source trace viewer and framework aimed towards performance and reliability analysis. Trace Compass can analyze Linux kernel data to show the frequency and state of CPUs over time, including interrupts, system calls, or userspace processes executing on each CPU. It can also display the state of each thread over time, as shown in Fig. 5 for the application threads. Building Trace Compass analyses specific to ROS 2 is left as future work; however, we can visualize timestamps of ROS 2 trace events on top of the existing analyses. From the ROS 2 trace data shown in Fig. 4, we know that the timer callback instance following the longest interval is at the 1.4 s mark with 107.8 ms. Finding the timestamps of the corresponding ROS 2 events in Trace Compass, we see that the thread was blocked waiting for CPU for 9.9 ms before the aforementioned callback instance. The thread then had to query the middleware for new messages (even if timers are strictly handled at the ROS 2 level) and finally call the overdue timer callback. In this example, a multi-threaded executor was used, with the number of threads being equal to the number of logical CPU cores by default. Since this was not the only application running on the system at that time, multiple processes and threads were competing for CPU, as can be observed using Trace Compass. Therefore, the executor settings could be tuned, or the executor could be replaced by another type of executor with features that better meet the requirements for this system, which could entail creating a new executor: this is an open problem in ROS 2. A multi-level analysis such as this one would not have been possible without collecting both userspace & kernel execution information, and analyzing the combined data, which

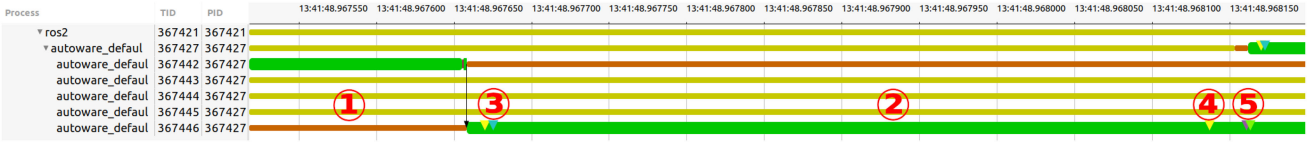

Fig. 5. State of ROS 2 application threads over time with timestamps of ROS 2 events from the `ros2_tracing` instrumentation displayed as small triangles on top of the thread state rectangle: ① thread waiting for CPU for 9.9 ms (orange), ② thread running (green), ③ event marking start of middleware query & wait for new messages, ④ event marking end of middleware query, and ⑤ `rclcpp_executor_execute` event followed shortly after by `callback_start` event for timer callback. This result was obtained by importing trace data collected from the Linux kernel and from the ROS 2 application using LTTng into Trace Compass [29]. The black arrow to the left of ③ represents the scheduling switch from one thread to another for a given CPU. Some less relevant threads were hidden.

TABLE III
EXPERIMENT PARAMETERS AND VALUES

Publishing rate (Hz)	100, 500, 1000, 2000
Message size (KiB)	1, 32, 64, 256
Quality of service	reliable only
DDS implementation	eProsima Fast DDS

current tools do not offer. The scripts and full instructions to replicate this example are available at.[1]

VI. EVALUATION

To validate that our proposed solution is compatible latency-wise with real-time systems, we evaluate the overhead of `ros2_tracing`, or specifically its instrumentation overhead. This is the time consumed by the instrumentation within the monitored process. When enabled, it directly affects these processes by adding latency.

Since the `ros2_tracing` tracepoints are placed along the message publication and reception pipeline, an easy way to capture the maximum overhead is to measure the time between publishing a message and when it is handled by the subscription callback. This is what we will do in the following.

A. Experiment Setup

We use the standard message-passing latency benchmark for ROS 2, performance_test [31], with a minimal configuration: one publisher node and one subscription node. We vary message size and publishing rate, since it is known that they affect middleware performance [19]. The parameter space is shown in Table III and is based on typical use-cases [32].

To reduce measurement variability, we follow common practice by using a kernel patched with the PREEMPT_RT patch (5.4.3-rt1), disabling simultaneous multithreading, and disabling power-saving features in the BIOS (dynamic frequency scaling, C-states, Turbo Boost, etc.). Further, we increase UDP buffers to 64 MB to ensure sufficient networking performance for larger messages. The experiment was run on an Intel i7-3770 (3.40 GHz) 4-core CPU, 8 GB RAM system with Ubuntu 20.04.2. All measurements are based on the ROS 2 Rolling distribution, in between the Galactic and Humble releases, which is the most recent version at this time. While Eclipse Cyclone DDS [24] is the default DDS implementation for the current ROS 2 release, Galactic, we have found eProsima Fast DDS [33] to be more stable for larger messages, and it is also the default for the upcoming release, Humble. We have therefore used Fast DDS.

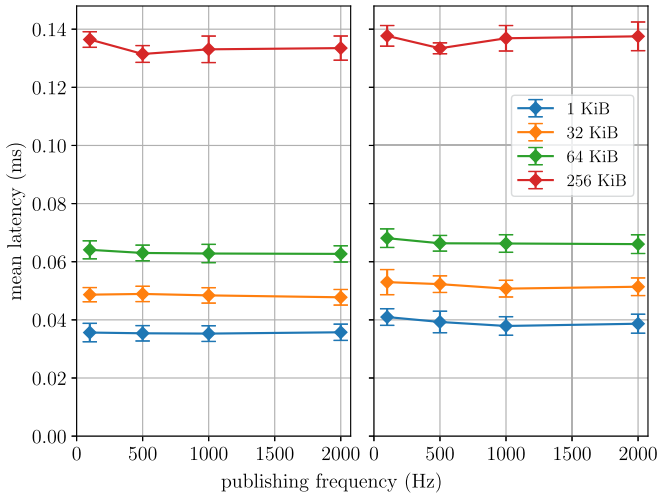

Fig. 6. Message latencies (avg. ± std.) without tracing (left) and with tracing (right).

For each combination in the parameter space, performance_test is run for 60 minutes and scheduled with the SCHED_FIDO real-time policy with the highest priority (99). To reduce outliers due to system initialization, the first 10 seconds of each recording are discarded. To determine the tracing overhead, we run the experiment once without any tracing enabled, and once with all tracepoints enabled. In practice, since not all tracepoints might be needed depending on the intended analysis, this represents the worst-case scenario.

The code and instructions to replicate this experiment are available at.[2]

B. Results and Discussion

Fig. 6 shows the individual average latencies without and with tracing, while Fig. 7 shows the absolute and relative latency overhead.

[1][Online]. Available: https://github.com/christophebedard/ros2_tracing-analysis-example

[2][Online]. Available: https://github.com/christophebedard/ros2_tracing-overhead-evaluation

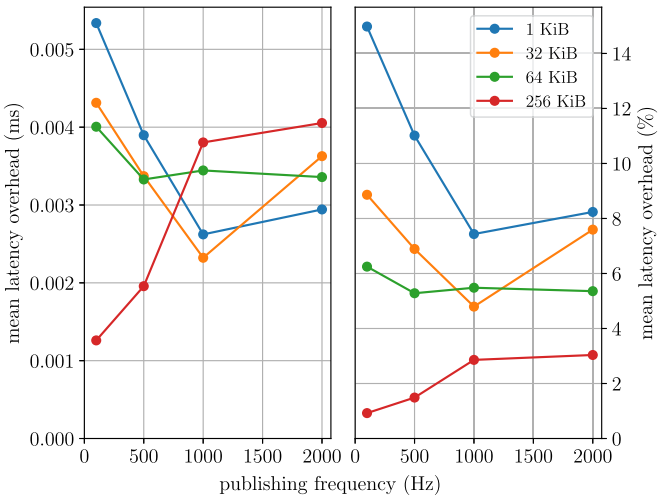

Fig. 7. Absolute (left) and relative (right) latency overhead results. The standard deviation of the difference between the two means is insignificant here.

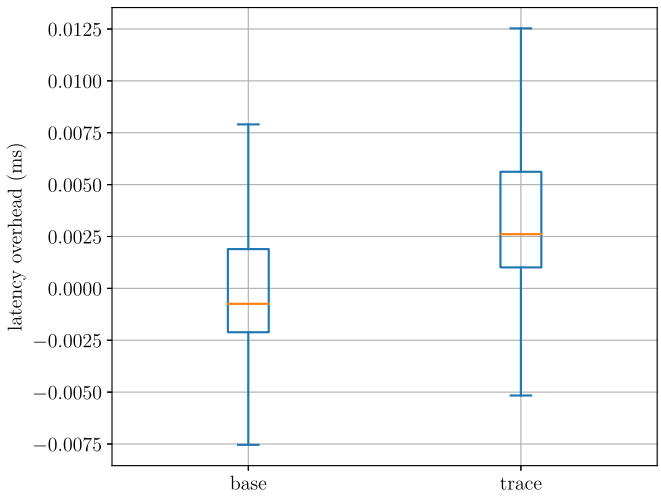

Fig. 8. Aggregated latency overhead and variation without tracing (left) and with tracing (right). Latency values have been individually normalized to zero mean based on the latencies without tracing, showing overhead and variation. Note that the left mean is very slightly below zero due to the additional imbalance caused by the variation in publishing frequency.

First, as expected, the mean latency values increase with the message size, and the relative latency overhead values decrease with the message size. There is no significant decrease in latency as the publishing frequency increases; this behavior was however much more noticeable before disabling power-saving features through the BIOS. We would also expect the CPU overhead to be the same for all message sizes and publishing frequencies, since it is, in theory, a constant overhead for message publication and reception. However, it can be seen that this is not the case, and instead, the absolute overhead is larger at small frequencies. This is somewhat puzzling, and it would certainly merit further experiments. However, due to the overall small effect, we are approaching a range where cache effects in the CPU or other non-deterministic factors come into play. Since the overhead values are overall fairly close, the overhead does not seem to be related to any of the experiment parameters, and the absolute values are well within acceptable ranges, we consider the requirements set out for `ros2_tracing` to be fulfilled. Additionally, these absolute latency overhead results are within one order of magnitude of the results that [7] presented: since there are 10 tracepoints in the publish-subscribe hot path (see Table II), the overhead should therefore be $10 \cdot 158$ ns $=$ 0.00158 ms, which is indeed comparable.

Since most practical systems use a mixture of message sizes and frequencies, we also analyze the overhead by aggregating it over all experiment runs. For each combination of message size and publishing frequency, we use the two sets of latencies (i.e., without tracing and with tracing, represented in Fig. 6), and subtract the mean of the no-tracing set from all latencies. By aggregating the latency differences for all combinations, we obtain two sets of latency overheads, which are represented in Fig. 8 (without tracing and with tracing, respectively). Note that the aggregate overhead is more strongly influenced by the higher publication frequencies, since more messages are sent in the same time frame. The mean overhead is thus 0.0033 ms, with 50% of the data between 0.0010 ms and 0.0056 ms.

No measurement system can be entirely without overhead—however, we think that these results show that `ros2_tracing` has very low overhead, which is acceptable for most target situations for ROS 2. It is certainly lower than known alternatives as presented in Table I.

Nonetheless, for very busy systems or very particularly CPU-constrained platforms, overheads might add up enough to impact the messaging performance. For such situations, there are two potential optimization options: First, about half of the tracepoints would be sufficient for basic information, and, second, tracing can be selectively enabled on only some of the processes, instead of all ROS 2 nodes.

VII. Future Work

Many improvements and additions could be made to `ros2_tracing`. While the RPC pattern is not used as much in real-time applications, instrumentation could be added to support services and actions, with the latter being services with optional progress feedback. Furthermore, `ros2_tracing` does not gather information about object destruction, e.g., if a publisher or a subscription is destroyed during runtime. This is because it does not fit with design guidelines of real-time safety-critical systems, where the system is usually static once it enters its runtime phase. Nonetheless, complete object lifetime information could be gathered. Middleware implementations could also be instrumented to provide lower-level information on the handling of messages. Instrumenting `rclc` [34], [35] would also be interesting. It is a client library written in C with a deterministic executor aimed at ROS 2 applications on memory-limited real-time platforms such as microcontrollers. As for the usability tools, as mentioned previously, the orchestration tools could be improved, when native support for remote or coordinated multi-host orchestration gets added to the ROS 2 launch system.

While the overall approach taken for `ros2_tracing` is primarily aimed at offline monitoring and analysis, the instrumentation itself could be leveraged for online monitoring. For

example, the LTTng live mode could be used to do online processing of the trace data. Another backend could also be added for a tracer that better supports online monitoring. There could also be other default backends for other operating systems, like QNX, which is often used for real-time as well.

In future work, we plan on building on the `ros2_tracing` instrumentation and tools to analyze the internal workings of ROS 2. For example, as mentioned in Sections II and V, the determinism and efficiency of the ROS 2 executors could be analyzed and compared to proposed alternatives. The ROS 2 instrumentation could of course also be used in conjunction with the LTTng built-in userspace and kernel instrumentation, as demonstrated in Section V. For example, to verify real-time systems, unwanted runtime phase dynamic memory allocations could be detected by combining lifecycle node state information and the LTTng `libc` memory allocation tracepoints. Trace Compass could also be used to provide analyses and views specific to ROS 2.

VIII. Conclusion

Testing and debugging robotic systems, based on recordings of high-level messages, does not provide sufficient information on the computation performed, to identify causes of performance bottlenecks or other issues. Existing methods target very specific problems and thus cannot be used for multipurpose analysis. They are also not suitable for real-world real-time applications, because of their high overhead or poor usability.

We presented `ros2_tracing`, a framework with instrumentation and flexible tools to trace ROS 2. The extensible multipurpose low-overhead instrumentation for the ROS 2 core allows collecting execution information to analyze any ROS 2 system. The tools promote usability through their integration with the ROS 2 orchestration system and other usability tools. Our experiments showed that the message latency overhead it introduces is in an acceptable range for real-time systems built on ROS 2. These tools enable testing and debugging ROS 2 applications based on internal execution information, in a manner that is compatible with real-time applications and real-world development processes. Analyzing the combined trace data, from a ROS 2 application and the operating system, can help find the cause of performance bottlenecks and other issues. We plan on leveraging `ros2_tracing` in future work to analyze the internal handling of ROS 2 messages.

References

[1] M. Quigley *et al.*, "ROS: An open-source robot operating system," in *Proc. ICRA Workshop Open Source Softw.*, 2009.

[2] Open Robotics, "ROS 2." [Online]. Available: https://docs.ros.org/en/rolling/

[3] J. Perron, "VIPER: Volatiles investigating polar exploration rover," in *Proc. ROS World 2021. Open Robot.*, Oct. 2021. [Online]. Available: https://vimeo.com/649657650

[4] M. Quigley, B. Gerkey, and W. D. Smart, "Debugging robot behavior," in *Programming Robots With ROS*. Newton, MA, USA: O'Reilly, 2015, pp. 375–396.

[5] A. Afzal, C. L. Goues, M. Hilton, and C. S. Timperley, "A study on challenges of testing robotic systems," in *Proc. IEEE 13th Int. Conf. Softw. Testing, Validation Verification*, 2020, pp. 96–107.

[6] B. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed., London, U.K.: Pearson, 2020.

[7] M. Gebai and M. R. Dagenais, "Survey and analysis of kernel and userspace tracers on linux: Design, implementation, and overhead," *ACM Comput. Surv.*, vol. 51, no. 2, pp. 1–33, 2018.

[8] M. Desnoyers and M. R. Dagenais, "The LTTng tracer: A low impact performance and behavior monitor for GNU/Linux," in *Proc. Ottawa Linux Symp.*, 2006, pp. 209–224.

[9] B. Poirier, R. Roy, and M. Dagenais, "Accurate offline synchronization of distributed traces using kernel-level events," *ACM SIGOPS Operating Syst. Rev.*, vol. 44, no. 3, pp. 75–87, 2010.

[10] L. Gelle, N. Ezzati-Jivan, and M. R. Dagenais, "Combining distributed and kernel tracing for performance analysis of cloud applications," *Electronics*, vol. 10, no. 21, 2021, Art. no. 2610.

[11] T. Blass, A. Hamann, R. Lange, D. Ziegenbein, and B. B. Brandenburg, "Automatic latency management for ROS 2: Benefits, challenges, and open problems," in *Proc. IEEE 27th Real-Time Embedded Technol. Appl. Symp.*, 2021, pp. 264–277.

[12] K. Nishimura, T. Ishikawa, H. Sasaki, and S. Kato, "RAPLET: Demystifying publish/subscribe latency for ROS applications," in *Proc. IEEE 27th Int. Conf. Embedded Real-Time Comput. Syst. Appl.*, 2021, pp. 41–50.

[13] B. Gregg, "Linux performance." [Online]. Available: https://www.brendangregg.com/linuxperf.html

[14] I. Lütkebohle, "Determinism in ROS—or when things break/sometimes/and how to fix it...," in *Proc. ROSCon Vancouver Open Robot.*, 2017. [Online]. Available: https://doi.org/10.36288/ROSCon2017-900789

[15] Bosch Corporate Research, "ROS 1 tracetools." [Online]. Available: https://github.com/boschresearch/ros1_tracetools

[16] C. Bédard, "Message flow analysis for ROS through tracing," 2019. [Online]. Available: https://christophebedard.com/ros-tracing-message-flow/

[17] B. Gerkey, "Why ROS 2?," [Online]. Available: https://design.ros2.org/articles/why_ros2.html

[18] J. Kay, "Proposal for implementation of real-time systems in ROS 2," [Online]. Available: https://design.ros2.org/articles/realtime_proposal.html

[19] T. Kronauer, J. Pohlmann, M. Matthé, T. Smejkal, and G. Fettweis, "Latency analysis of ROS2 multi-node systems," in *Proc. IEEE Int. Conf. Multisensor Fusion and Integration for Intell. Syst.*, 2021.

[20] Z. Jiang *et al.*, "Message passing optimization in robot operating system," *Int. J. Parallel Program.*, vol. 48, no. 1, pp. 119–136, 2020.

[21] D. Casini, T. Blaß, I. Lütkebohle, and B. B. Brandenburg, "Response-time analysis of ROS 2 processing chains under reservation-based scheduling," in *Proc. 31st Euromicro Conf. Real-Time Syst.*, 2019, pp. 1–23.

[22] S. Rivera, A. K. Iannillo, S. Lagraa, C. Joly, and R. State, "ROS-FM: Fast monitoring for the robotic operating system (ROS)," in *Proc. 25th Int. Conf. Eng. Complex Comput. Syst.*, 2020, pp. 187–196.

[23] J. Peeck, J. Schlatow, and R. Ernst, "Online latency monitoring of time-sensitive event chains in safety-critical applications," in *Proc. Des., Autom. Test Europe Conf. Exhibit.*, 2021, pp. 539–542.

[24] Eclipse Foundation, "Cyclone DDS," [Online]. Available: https://github.com/eclipse-cyclonedds/cyclonedds

[25] G. Biggs and T. Foote, "Managed nodes," [Online]. Available: https://design.ros2.org/articles/node_lifecycle.html

[26] P.-M. Fournier, M. Desnoyers, and M. R. Dagenais, "Combined tracing of the kernel and applications with LTTng," in *Proc. Linux Symp.*, 2009, pp. 87–93.

[27] ROS 2 Real-Time Working Group, "Reference system," [Online]. Available: https://github.com/ros-realtime/reference-system

[28] "Tracetools_analysis," [Online]. Available: https://gitlab.com/ros-tracing/tracetools_analysis

[29] Eclipse Foundation, "Trace compass," [Online]. Available: https://www.eclipse.org/tracecompass/

[30] M. Côté and M. R. Dagenais, "Problem detection in real-time systems by trace analysis," *Adv. Comput. Eng.*, vol. 2016, 2016, Art. no. 9467181.

[31] Apex.AI, "Performance_test." [Online]. Available: https://gitlab.com/ApexAI/performance_test

[32] M. Reke *et al.*, "A self-driving car architecture in ROS2," in *Proc. Int. SAUPEC/RobMech/PRASA Conf.*, 2020, pp. 1–6.

[33] eProsima, "Fast DDS," [Online]. Available: https://github.com/eProsima/Fast-DDS

[34] J. Staschulat, I. Lütkebohle, and R. Lange, "The RCLC executor: Domain-specific deterministic scheduling mechanisms for ROS applications on microcontrollers: Work-in-progress," in *Proc. Int. Conf. Embedded Softw.*, 2020, pp. 18–19.

[35] J. Staschulat, R. Lange, and D. N. Dasari, "Budget-based real-time executor for micro-ROS," 2021, *arXiv:2105.05590*.